PaN-Finder Deliverable 2.b:

Report on current system architecture

Table of Contents

1 System Components....3
2 Data Collectors....5
2.1 ESRF....5
2.2 Output file....8
2.3 Future improvements....9
2.3.1 Lab and User Reports, ELNs....10
2.3.2 Related publications....10
2.3.3 Aggregators Record....10
3 Data Ingestion....10
3.1.1 Entry (document) ingestion....11
3.1.2 Chunk Ingestor....11
3.1.3 Summary Ingestor....13
3.1.4 Filter Ingestor....13
3.1.5 Numeric Filter Ingestor....15
3.1.6 Filter Value Converter....15
3.1.7 Filter Enricher....16
4 Backend (Engine)....17
4.1 Search....17
4.1.1 Search....17
4.1.2 Explain....20
4.1.3 Structured....20
4.2 Document....20
4.2.1 Raw....21
4.2.2 Document....22
4.3 Feedback....23
4.3.1 Submit....23
4.4 Session....23
4.4.1 Create....23
5 Frontend (User Interface)....23
5.1.1 Results List....26
5.1.2 Entry detailed view....28
5.1.3 Query details....30
6 Database....31

CC BY-SA

6.1 Application databases....31
6.1.1 Document....31
6.1.2 Chunk....32
6.1.3 Facility....33
6.1.4 Filter....33
6.1.5 Filter Description....34
6.1.6 Filter Key....34
6.1.7 Statistic....35
6.1.8 Feedback....35
6.2 Benchmark databases....36
6.2.1 Test Pairs....36
6.2.2 Benchmarks Run....37
6.2.3 Benchmarks Run Test....37
6.2.4 Test Pair Metrics Values....39
6.2.5 Run Metrics Values....39
7 Benchmarks....40
8 Funding....41
9 Licensing....41

CC BY-SA

1 System Components

The PaN-Finder system is compose by the following 6 main components:

1. Data collectors
2. Data ingestor
3. Backend or Engine
4. Frontend or UI (User interface)
5. Database
6. Benchmarks

Each component is described in depth in the related sections below.

The overall architecture and the data flow is visible in the figure 1.

In figure 1, it is clearly visible that the system can be divided in three functional sections: the dtaa collection and ingestion, the backend and frontend providing service to the users and the scientific community and lastly, the benchmarking system which is used to test the system performances and accuracy.

The same figure also shows the data flow. The red arrows highlight the data flow during the ingestion process. The blue ones indicate the data flow during user interaction, while the green ones show the data flow during benchmarking.

CC BY-SA

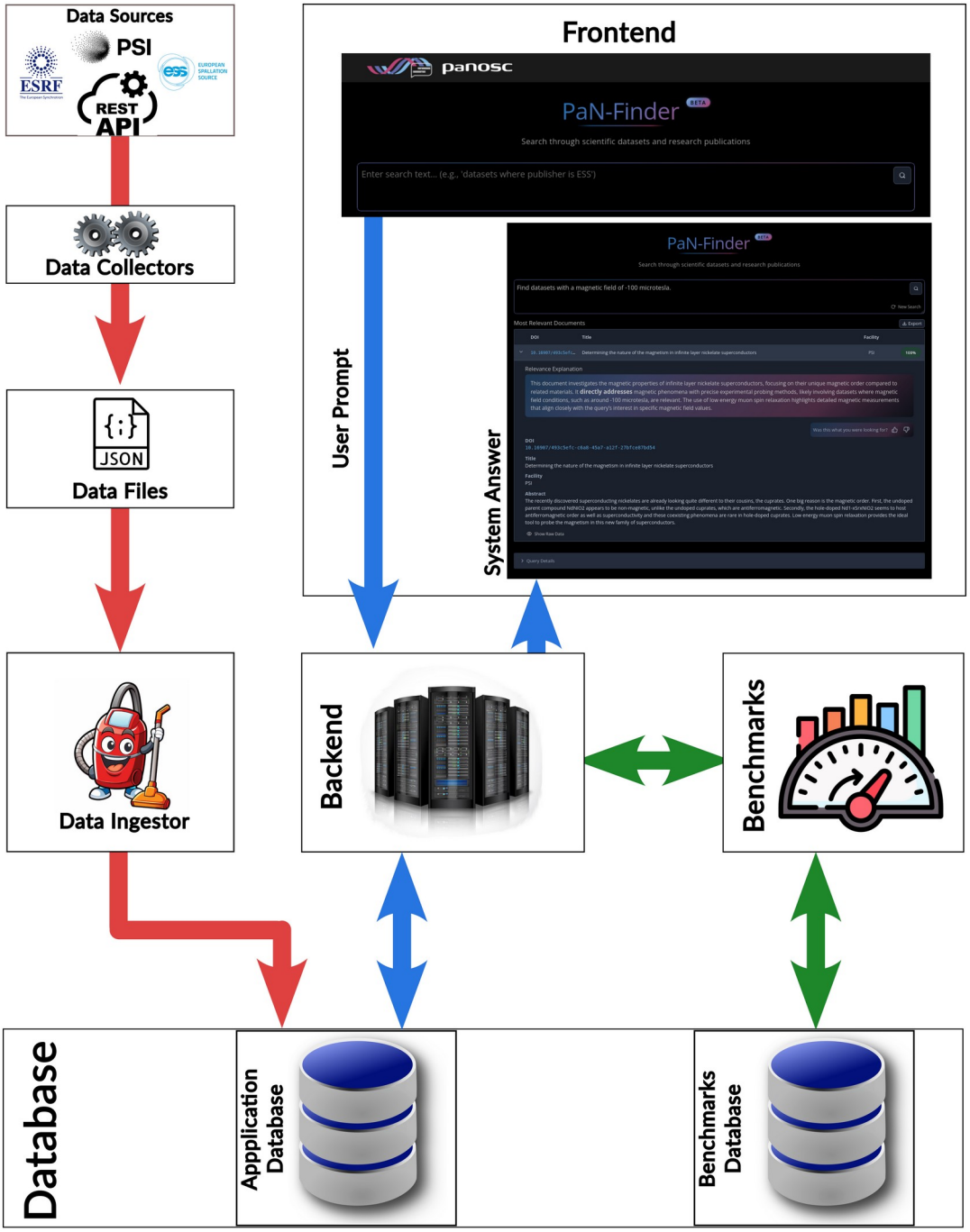

Figure 1: PaN-Finder system architecture and data flow

CC BY-SA

2 Data Collectors

Data collectors are the scripts responsible for retrieving all the information related about the open data available at each data provider (facility). They interact with the data catalogue API available at each data provider.
Each data provider requires a dedicated set of scripts to address the differences in system architectures, API endpoints and also data structure present across data providers. We decided to follow such philosophy in order to maximize the information collected and capture the information variety across the multiple data providers, while minimizing the software development.

The scripts are included in the deliverable repository under the folder task-1/scripts. The files are named according to the following convention:

oscars_pan_finder_collect_<facility_name>_<functionality>.<ext>

where:

- *<facility_name>* is the facility acronym, like ESRF, PSI or ESS
- *<functionality>* is a brief explanation of the script functionality
- *<ext>* the language or environment the script needs to run, like py or bash.

Each facility requires unique features, but overall the dedicated set of scripts share the same logic. In the next section, we illustrate the scripts used to collect information from ESRF which is the most complete at the time of this writing and well represents the overall methodology used to collect the information across facilities.

2.1 ESRF

Data collection from ESRF is organized in five different scripts:

- *oscars_pan_finder_collect_esrf_data.bash*
  This is a bash script that orchestrate and run the other scripts in the correct order and produce a single json script containing all the information collected
- *oscars_pan_finder_collect_esrf_panosc_documents.py*
  This python script calls the ESRF PaNOSC endpoint and retrieve all the PaNOSC entries. It does so in smaller batches not to overload the API and also it checks that the number of entries is consistent with the number provided by the API itself. The output is save in a file with the following name *oscars_pan_finder_esrf_panosc_documents.json.*

 CC BY-SA

- *oscars_pan_finder_collect_esrf_entries_from_panosc_list.py*
  This python script loads the full list produced from the PaNOSC list and collect the remaining information from the ESRF data portal API leveraging the next script.
- *oscars_pan_finder_collect_esrf_entry.py*
  This python script collects all the information available from the ESRF data portal API regarding a specific public data entry identified by a registered DOI.
- *oscars_pan_finder_settings_esrf.py*
  This script is a utility file where all the common settings and options are stored and shared across all the python scripts listed above

Figure 2 illustrates how the data collection is orchestrated and performed by the different scripts for ESRF (European Synchrotron Radiation Facility, https://www.esrf.fr/home.html).
Script *oscars_pan_finder_collect_esrf_panosc_documents* leverage the PaNOSC API present at ESRF to retrieve all the open data records available at the facility. Script *oscars_pan_finder_collect_esrf_entries_from_panosc_list* load such list and, for each entries, runs *oscars_pan_finder_collect_esrf_entry* which requests from the relevant endpoint the DataCite record, the investigation (which correspond to the session or the beam time allocated to the users to run their experiments), the datasets collected in such session, the samples listed by the users, the list of users and if present any user report that users uploaded after the session. Datasets, samples, users and user report are collected only if the session is not under embargo, given that ESRF releases right away information about the session and also associates a DOI.

In figure 2 are visible all the endpoint and the corresponding entry the information is inserted under in the output file. The output file is named *esrf_document_<DOI>.json* where <DOI> is the document doi registered with DataCite properly formatted to include only underscores.

The bash script coordinates running these scripts and, at the end, collect each individual json file in a single one named *oscars_pan_finder_esrf_data_<timestamp>.json* where *<timestamp>* is the timestamp when the file has been created in the format *YYYYmmddHHMMSS.*

CC BY-SA

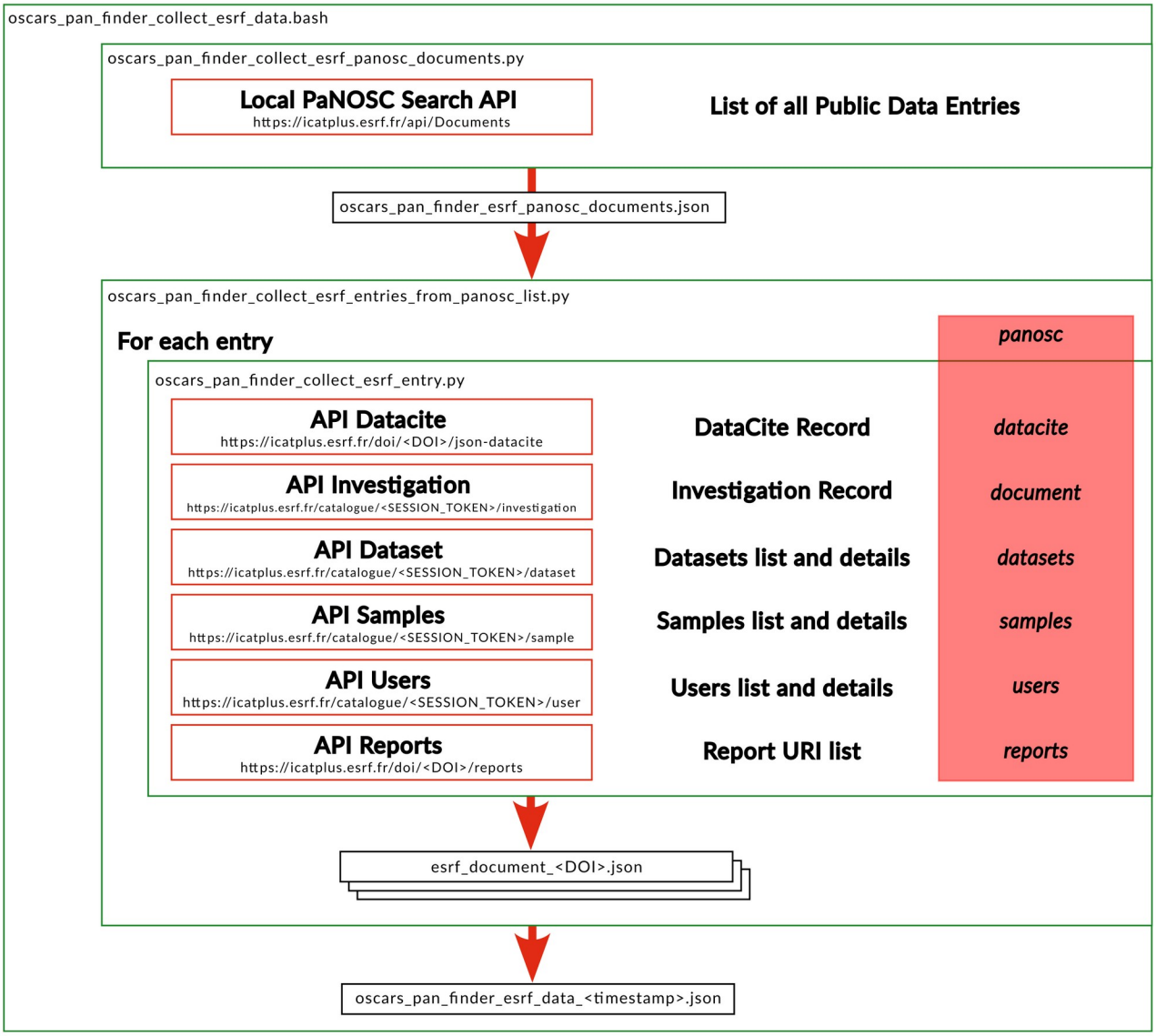

Figure 2: ESRF collection scripts, API endpoints and information collected

CC BY-SA

2.2 Output file

The output file is in json format and for ESRF data contains the following sections:

- *panosc*: the record retrieved through the PaNOSC API endpoint
- *datacite*: the record retrieved through the datacite API endpoint
- *document*: the main data retrieved for this entry through the data provider data catalogue API. For ESRF data the document corresponds to the proposal session which is the allocated time that users have at the instrument to conduct all their data acquisition. If a proposal has multiple sessions, it will have multiple documents.
- *datasets*: the list of datasets that have been acquired under this sessions.
- *samples*: the list of samples that have been declared in this entry and the user intended to use or used during the data acquisition.
- *users*: list of users that were allowed on site or granted access to the data or are linked in any way with the proposal.

CC BY-SA

- *reports*: list of URLs for reports that the users has compiled for the specific session and its outcome.

An example of such file is visible in figure 3. Some of the section are collapsed for easy viewing.


```
▶ panosc:                { pid: "651037210", isPublic: true, type: "MA", … }
▶ datacite:              { doi: "10.15151/ESRF-ES-651037210", url: "https://doi.esrf.fr/10.15151/ESRF-ES-651037210", publicationYear: "2025", … }
▼ document:
    name:                "MA-5093"
    startDate:           "2022-02-23T08:00:00.000+01:00"
    endDate:             "2022-02-25T08:00:00.000+01:00"
    id:                  651037210
    doi:                 "10.15151/ESRF-ES-651037210"
    title:               "Elucidating the complex rheological behavior of industrially relevant ceramic-based stereolithography pastes"
    visitId:             "MA/5093 ID02 23-02-2022/25-02-2022"
    releaseDate:         "2025-02-25T08:00:00.000+01:00"
  ▶ summary:             "We aim to understand the rheological behaviour of an industrially relevant ceramic-based stereolithography (SLA) photocurabl…la
  ▶ parameters:          { __fileCount: "4144", __volume: "283791067940", __datasetCount: "6", … }
  ▶ instrument:          { name: "ID02", id: 199806206, instrumentScientists: [] }
    investigationUsers:  []
▶ datasets:              (6)[ {…}, {…}, {…}, {…}, {…}, {…} ]
▶ reports:               [ "http://ftp.esrf.fr/pub/UserReports/97764_B.pdf" ]
▼ samples:
  ▶ 0:                   { id: 660150187, name: "3Ycom", modTime: "2022-01-19T14:04:45.609+01:00", … }
  ▶ 1:                   { id: 660150162, name: "3Yhome", modTime: "2022-01-19T14:04:45.519+01:00", … }
  ▶ 2:                   { id: 706911070, name: "Align", modTime: "2022-02-23T11:58:41.657+01:00", … }
  ▶ 3:                   { id: 660150141, name: "Alucom", modTime: "2022-01-19T14:04:45.406+01:00", … }
  ▶ 4:                   { id: 708484044, name: "Rheo", modTime: "2022-02-24T19:27:11.395+01:00", … }
  ▶ 5:                   { id: 708630859, name: "RheoCap", modTime: "2022-02-24T21:59:24.275+01:00", … }
  ▶ 6:                   { id: 707374734, name: "Rondelles", modTime: "2022-02-23T19:25:53.364+01:00", … }
  ▶ 7:                   { id: 708737672, name: "Static", modTime: "2022-02-24T23:39:18.192+01:00", … }
▼ users:
  ▶ 0:                   { fullName: "William CHEVREMONT", email: "william.chevremont@esrf.fr", familyName: "Chevremont", … }
  ▶ 1:                   { fullName: "Sylvain FOURNIER", email: "sylvain.fournier@insa-lyon.fr", familyName: "Fournier", … }
  ▶ 2:                   { fullName: "Pablo GRIFFITHS", email: "pablo.griffiths@insa-lyon.fr", familyName: "Griffiths", … }
  ▶ 3:                   { fullName: "Jérôme CHEVALIER", email: "jerome.chevalier@insa-lyon.fr", familyName: "Chevalier", … }
  ▶ 4:                   { fullName: "GUILHEM BAEZA", email: "guilhem.baeza@insa-lyon.fr", familyName: "Baeza", … }
```


Figure 3: ESRF data entry collected for the PaN-Finder

2.3 Future improvements

Since the beginning of the project, while working both on building the body of knowledge and developing the application, we have already identified multiple improvements that we can implement to enrich the information collected and increase the likelihood to increase the relevancy of the results returned for each user query.

 BY-SA

Such improvements are discussed in the next sections and will be developed in the near future if the interest, demand and support for the project continues. So far we just have explored the potential and started exploring the feasibility. The technical details and implementation still needs to be investigated.

2.3.1 Lab and User Reports, ELNs

Some entries might have lab reports, user reports or the entire electronic lab notebook available for public download. Independently from the format provided, we have discovered that such documents contains additional information that can be leverage by the PaN-Finder to hone in on the results that the user is looking for. With the aid of NLPs and LLM tools is now possible to extract, condense and summarize such information and include it in the body of knowledge that the PaN-Finder leverages.

2.3.2 Related publications

Related publications are another source of information that can be included to enrich the body of knowledge related to each entry. Leveraging services present at each facility, together with the new portal OpenAlex (https://openalex.org/), for each open data entry, we are confident that we can retrieve the publications that used or mentioned such data. Using the same workflow presented for the lab and user report, we can extract and ingest additional information for each entry, achieving the objective to expand the body of knowledge the PaN-Finder can work on.

2.3.3 Aggregators Record

In our quest of exploring how to expand and improve the data collection, we realize that organizations aggregating and cataloging public data entry, like B2Find, even if they are presenting a sub-set of the information present at the data provider, they provide them in different terms, using different words. WE are confident that such nuances and subtle differences can bring a positive contribution to the body of knowledge of the project.

3 Data Ingestion

The data ingestion is the action to digest, curate and fit the information collected for each entry into the different components that the PaN-Finder uses to select the relevant entries and score them.
The ingestion process is visualized in figure 4. It follows the following steps which are discussed in the related sections:

- entry or document ingestion
- chunk ingestor
- summary ingestor

CC BY-SA

- filter ingestion
- numeric filter ingestor
- filter value converter
- filter enricher

The ingestion process currently included in the repository is tailored to the data that have been collected for the PaN-Finder scope. If users plan to adopt the PaN-Finder tool for other purposes, they need to revise and adapt the ingestion to the body of knowledge they intend to have the PaN-finder act upon.

The process has been divided in multiple steps to maintain modularity, keep separation of functionality and simplify the code for required to implement each step.

3.1.1 Entry (document) ingestion

The Entry Ingestion has the responsibility to read each entry information from the json file collected from the data sources and insert it in the database under the table *document*. The record inserted in the database contains the doi, title and abstract or description, and also the full raw json representation of the entry as read from the file. Only public data with a valid DOI are ingested. If a DOI is not present, the entry is discarded. An entry was originally referred as document and document is still used in the code. The code for this portion of the ingestion is accessible in file *ingestor/src/document_ingestor.py*.

3.1.2 Chunk Ingestor

In the Chunk Ingestor step, we create the chunks for all the documents ingested in the previous step. Each document is loaded, the title and text is combined and fed to the chunking algorithm which divides the text in shorter sequences called chunks. Each chunks is encoded in a vector representation called embedding by a dedicated neural network. Each pairs of chunk and corresponding embedding is than saved in the database under the *chunk* table. The chunking is performed by the class *StastisticalChunker* offered by the python library *semantic_chunkers* under the *chunkers* sub package. The *StatisticalChunker* requires the use of an encoder, which we currently hard-coded to the *sentence-transformers/all-MiniLM-L12-v2* available through the Hugging Face platform (https://huggingface.co/) and accessible through the *HuggingFaceEncoder* class of the *semantic_router* python library *encoders* sub-package. The encoding of each chunks is performed using the *SentenceTransformer* class of the *sentence_transformer* python library. Such class requires the model of the encoder that is used in the encoding. By design, this encoder is set in the application configuration, so it can be quickly changed facilitating parameter exploration and application fine tuning. The current settings see the encode set to *all-MiniLM-L12-v2*, which is the same used in the chunking algorithm.

 CC BY-SA

At the end of this step all the entries with a title and description have one or more chunks with their related embedding saved in the database.
The code implementing this step is saved in the file ingestor/src/chunk_ingestor.py.

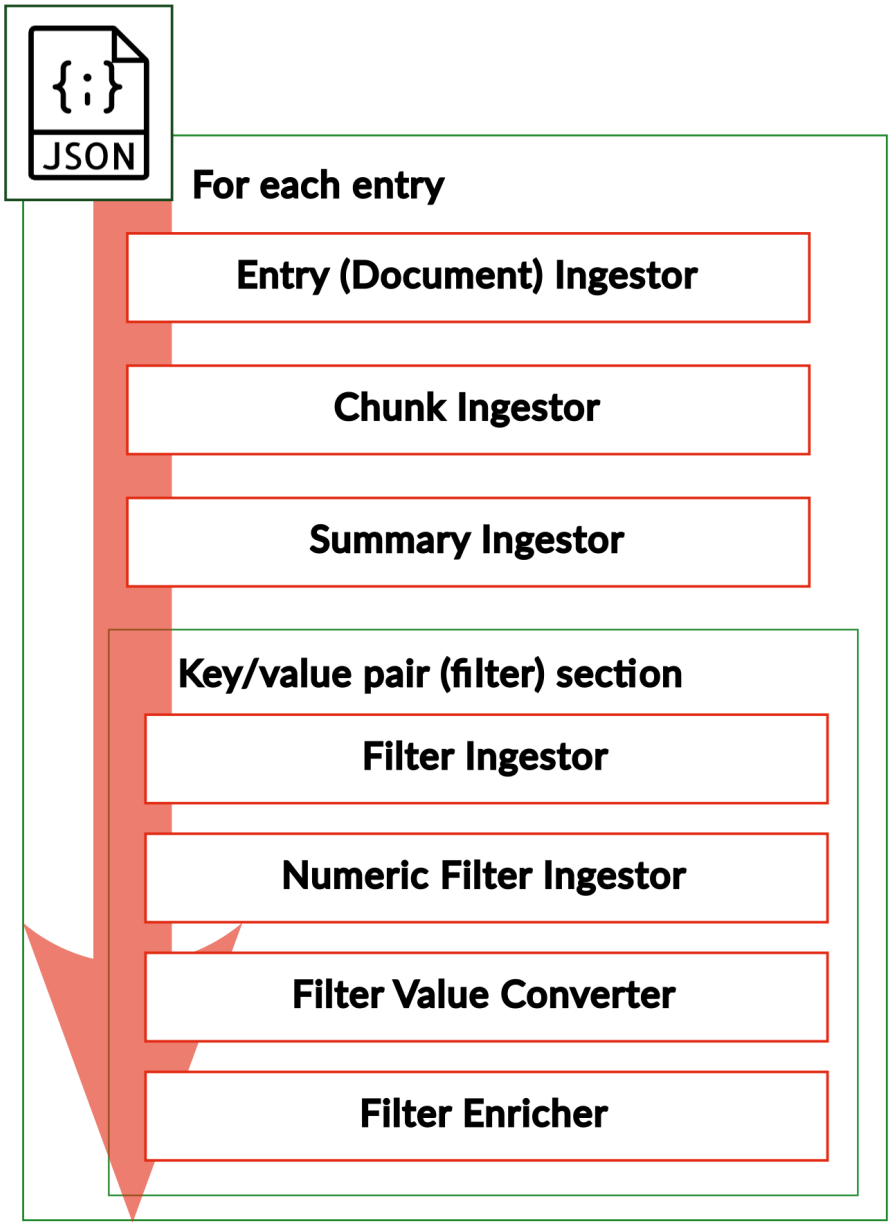

Figure 4: Data Ingestion Process

CC BY-SA

3.1.3 Summary Ingestor

The Summary Ingestor steps creates a summary from title and description using an LLM. It loads all the entries and for each one of them, it extract and combine title and description and submit the request for a 100 words summary to an LLM. The summary returned is than encoded in an embedding and save both of them in the *summary* fields of the *document* table.
The code allows to use both LLM models running locally or on cloud providers. At the time of this writing, we are using a *gpt-4.1-mini* model running on Azure cloud infrastructure. Each summary is created submitting to the LLM the user prompt shown in figure 6, in which we have substituted title and description of the entry, together with system prompt v1.0.9 available in file *benchmark/prompts/gpt-4.1-mini/1_0_9.md*.

```
prompt_system = (
    "You are a helpful assistant that summarizes scientific texts in <= 100 words. "
    "If you cannot summarize, return the provided title. Output only the summary text."
)
prompt_user = f"Title: {title}\n\nText: {text}"
```

Figure 5: Summary user prompt

The system prompt is one of the parameters that we can act when tuning further the application. Potentially, we could move the user prompt to configuration, in order to make it configurable and use it also in the application fine tuning.
The embedding of the summary is calculated exactly as in the chunk step. We use *SentenceTransformer* class of the *sentence_transformer* python library with a configurable embedding model. The current settings have the encoding model set to *all-MiniLM-L12-v2*, as in the previous cases.
The code for this step can be found in file *ingestor/src/summary_ingestor.py*.

3.1.4 Filter Ingestor

The Filter Ingestor step extract key-value pairs from the information available for each entry and store them in the database under the *filter* table. The code takes the full nested json structure for the entry, flatten it out, obtaining a list of keys and values pairs which are saved in the *filter* table as tuples (entry id, key, value).

To illustrate better the output of this step, when the following entry is provided:

CC BY-SA

```yaml
datasets:
  - owner: "Max Novelli"
    contactEmail: "max.novelli@ess.eu"
    sourceFolder: "/ess/data/v20/2019/legacy/raw/v20/14CD3N"
    size: 55475106
    type: "raw"
    keywords: [v20, neutron]
    description: "V20 data"
    datasetName: "open beam, WFM Slits 0.2x50"
    license: "CC-BY-4.0"
    isPublished: true
    createdAt: "2019-09-04T11:11:55.917Z"
    updatedAt: "2023-08-23T12:42:44.726Z"
    scientificMetadata:
      chopper_1_radius:
        value: 350
        unit: "mm"
        valueSI: 0.35
        unitSI: "m"
      chopper_1_name:
        value: "Airbus, Source Chopper, ESS Pulse, Disc 1"
        unit: ""
      chopper_2_radius:
        value: 350
        unit: "mm"
        valueSI: 0.35
        unitSI: "m"
      chopper_2_name:
        value: "Airbus, Source Chopper, ESS Pulse, Disc 2"
        unit: ""
```


Internally, it is converted to the following list of individual key, value pairs:


```yaml
owner: "Max Novelli"
contactEmail: "max.novelli@ess.eu"
sourceFolder: "/ess/data/v20/2019/legacy/raw/v20/14CD3N"
size: 55475106
type: "raw"
keywords: "v20, neutron"
description: "V20 data"
datasetName: "open beam, WFM Slits 0.2x50"
license: "CC-BY-4.0"
isPublished: true
createdAt: "2019-09-04T11:11:55.917Z"
updatedAt: "2023-08-23T12:42:44.726Z"
scientificMetadata.chopper_1_radius.value: 350
scientificMetadata.chopper_1_radius.unit: "mm"
scientificMetadata.chopper_1_radius.valueSI: 0.35
scientificMetadata.chopper_1_radius.unitSI: "m"
scientificMetadata.chopper_1_name.value: "Airbus, Source Chopper, ESS Pulse, Disc 1"
scientificMetadata.chopper_1_name.unit: ""
scientificMetadata.chopper_2_radius.value: 350
scientificMetadata.chopper_2_radius.unit: "mm"
scientificMetadata.chopper_2_radius.valueSI: 0.35
scientificMetadata.chopper_2_radius.unitSI: "m"
scientificMetadata.chopper_2_name.value: "Airbus, Source Chopper, ESS Pulse, Disc 2"
scientificMetadata.chopper_2_name.unit: ""
```


CC BY-SA

All this entries are saved in the filter table and marked as “explicit” as they have extracted directly from the each entry without any additional curation.

An additional task that this step performs is the creation of the key embedding. Each key is curated by removing selected characters and the embedding is calculated using the encoding model *all-MiniLM-L12-v2* through the *sentence_transformer* python library. Each key and embedding tuple is saved in the related database *filter_key* table. Only unique keys are saved in the database, so the ingestion step checks if the key is already present and proceeds with the insert only if it is a new key.

3.1.5 Numeric Filter Ingestor

This ingestion step performs two type of tasks: converts string to value and unit pairs and create a range entry. It iterates on all the key values saved in the previous step, check if the value matches a string pattern containing a quantity, which is a numeric value pared with an SI unit. In case that value is indeed a quantity, it separates the value from the units and create a new key entry which contains the tuple key, value still in string format, and the explicit SI unit. These new filter entries are marked as *derived*. An example of a key quantity with value “37.5 Celsius”, which produces a new derived key entry with value 37.5 and unit equal to “Celsius”.
When the value of the key matches a range pattern, like “20-40 K”, two new derived entries are created one for the minimum and one for the maximum value. For our example, if the key is “temperature”, the two new entries are “temperature.min” with value 20 and unit “Kelvin” and “temperature.max” with value “40” and unit “Kelvin”. As in the previous task, these entries are also marked as *derived*. The new entries are saved in the *filters* table. The embedding is also computed for the new min and max keys and inserted in the *filter_key* table.

3.1.6 Filter Value Converter

The filter value converter step is responsible for converting the “string” value ingested previously in the filter table and convert it to the correct type. We leverage directly the capabilities present in postgresql and test if the original string value can be converted to a boolean, a number or a timestamp. If the conversion is successful, the new typed values are saved in the relative column of original entry of the *filter* table.
In case the value is successfully converted to a numeric type, the logic proceed to check if the entry has also a unit, which has been previously extracted. In such case, we proceed to convert the numeric value together to the specified unit to the matching value and SI units. This new entries

CC BY-SA

3.1.7 Filter Enricher

The filter enricher step is tasked to add to the filter table specific information regarding the facility or entity and the instrument or beamline where the data has been acquired or produced. Each entry contains the information about the facility where the data was created and it is stored. The value of such information is inconsistent, so, to facilitate the search later on, we create filters with key *publisher* of type *derived* where with the original facility name extract from the data collected. We also create a second *derived* filter where the value is the full facility name. For example, if we encounter an entry containing the ESS as the publisher, we convert the acronym ESS to European Spallation Source. The new derived filters are saved in the *filter* table.

The second task performed in this step is to address the differences of naming standards between entries collected at neutron sources vs the ones that are collected at synchrotrons, regarding the experimental setup and equipment. In neutron source facilities they are called *instruments*, while in synchrotrons they are called *beamlines*. After some experimenting, we found that it is easier and more performant to address this discrepancy at ingestion time than later when the user preform a search. We implemented the necessary code to find any filter where the key is *instrument*, and add a new derived filter where the key is *beamline*. We also do the opposite from *beamline* to *instrument*. The final result is that each entry will have two entry in the filters table with the same value, one with key *instrument* and one with key *beamline*.

CC BY-SA

4 Backend (Engine)

The backend implements a REST api which allow to submit a search request with the related prompt, return the relevant results found, and provide the relevancy summary for the requested result. It also includes the functionalities to collect and store user feedback and provide the complete entry data that was collected from the facility providing the data and ingested. In the following sections, each endpoint is discussed in details within the session regarding the specific file their implementation can be found in.

4.1 Search

This group of endpoints are tasked with performing the search (by prompt or by components) and provide the relevancy explanation. Their implementation is grouped together in the `search.py` file under the server folder.

4.1.1 Search

The search endpoint (`https://pan-finder.panosc.ess.eu/search`) implements the functionality were the user submits the prompt for the search and receives a list of 20 results together with their relevancy score.
The user prompt is submitted as payload in a POST request. The backend receives the prompt and extract the 3 main components:

- *intention*, which should be the essence of what the user wants from the system
- *keywords*, which are a minimal list of relevant words that are used in exact match over the text of the entry
- *filters*, which are the conditions expressed by the user which will be applied to filter and match the entries during the search

The components extraction is performed leveraging an LLM in order to be able to overcame the variety of the natural language submitted by the user and how diverse is the user community. Currently, as for the ingestion, we are leveraging a dedicated instance of *gpt-4.1-mini* model running on Azure cloud infrastructure.

CC BY-SA

The output of this first operation is a json data structure containing the three components listed above. Submitting the following prompt which is one of the default examples currently available on the application:

> Look for research on magnetic diffuse scattering in CuMnO2 where the temperature is between 1.5 K and 300 K and the sample mass is 10,000.

The components extracted are the following:

- intention: magnetic diffuse scattering in CuMnO2
- keywords: magnetic, diffuse, scattering, CuMnO2
- filters

```
{
  "logic": "AND",
  "conditions": [
    {
      "name": "temperature",
      "operator": ">=",
      "value": 1.5,
      "unit": "K"
    },
    {
      "name": "temperature",
      "operator": "<=",
      "value": 300,
      "unit": "K"
    },
    {
      "name": "sample mass",
      "operator": "=",
      "value": 10000
    }
  ]
}
```

The intention is than converted to its encoding vector using the same library and settings as in the ingestion: s*entenceTransformer* class of the *sentence_transformer* python library with the *all-MiniLM-L12-v2* encoding model. The encondings for all the filters conditions' keys are also computed.

The extracted components are provided to the query builder, which builds a hybrid sql query that includes the 5 sub queries listed below. Each subquery computes a raw relevancy score between 1 and 0 for each entries returned.
The 5 sub-queries are:

CC BY-SA

1. *Similarity search*
   This subquery implements a similarity search between the encoding of the intention component and the text containing the title and the summary.
   The raw score is 1 when we have a perfect match, while 0 means no similarity at all.

2. *Chunk similarity*
   This subquery implements similarity search between the encoding of the intention component and each entry chunks.
   The raw score is the similarity score of the best matching chunk and it will be 1 for exact match or 0 for no similarity at all.

3. *Keywords match*
   This query performs full text search on all the entries for the keywords extracted from the user prompt.
   The raw score for each entry is proportional to the number of keywords found over the total number of keywords searched. A score of one means that all keywords have been found in the entry exactly in the same form.

4. *Full query match*
   This query finds all the entries that match the exact filters as provided by the users.
   The raw score is 1 if there is a match or 0 if there is no match.

5. *Partial query match*
   This query finds all the entries that match at least one of the filter independently from the logic indicated by the user.
   The raw score is the number of conditions matched over the total number of conditions, with 1 being the case when all the filter conditions are matched.

The hybrid query is run in the database and the list of relevant documents is returned. The results list includes the DOI of the entry, the overall score and the scores provided by each subquery. The overall score is the sum of each individual scores, while each is subquery score is the RFF score computed from the raw score, explained above, weighted by a weight defined in configuration. RFF is the acronym of Reciprocal Rank Fusion which is an algorithm used to combined multiple ranked lists. It eliminates the need for score normalization, it assigns higher scores to higher ranking documents and it is recommended as best practice in RAG system like our. The RFF formula requires a user defined weight k, The lower is the weight, they higher is the importance of the subquery. This implementation allows us to control the importance of each subquery in the final score. The best combination of weights is decided based on the results of the benchmarks and it is influenced by data we work on. During our exploration phase, we used a default value of 6 for all the weights.

CC BY-SA

Ultimately the search endpoint returns a list containing doi, title, facility and overall relevancy score. During the whole process, the endpoint provides information on which phase of the process is currently performing.

4.1.2 Explain

The explain endpoint (`https://pan-finder.panosc.ess.eu/search/explain`) accept the doi of one of the documents returned by the search, submit a request to our LLM model (which is the same used during the search to extract components from the user prompt) and provides an explanation for why the specific document is relevant given the user prompt.
The request submitted to the LLM model include the user prompt, the sql submitted, the complete data resulting from the search itself, together with the complete entry for the document targeted. The request is completed by a dedicated prompt explaining th etask to the LLM model. This prompt is part of the configuration of the PaN-Finder application. Every time it is changed, the benchmarks have to be run to test that the system is actually performing in a similar way if not better.
The output is streamed, so the user is receiving the explanation word by word as it is produced by the LLM model. This design allows for a more user friendly experience.

4.1.3 Structured

The structured endpoint (`https://pan-finder.panosc.ess.eu/search/structured`) allow users to submit a query already broken down in their three main components: intention, keywords and filters. It completely bypass the step where the user prompt is submitted to the LLM model to extract the components.
This endpoint has been implemented to allow user to control the query components directly or correct any LLM mistake or hallucination in the component extraction. The components are use them as they are to build the hybrid query.
The output of the endpoint is similar to the normal search. It is streamed and produce a list of 20 entries with the related relevancy score.

4.2 Document

This group of endpoints have been implemented to provide access to each entry metadata collected and stored in the PaN-Finder dataset. The implementation of these endpoints is contained in the `document.py` file under the server folder.

CC BY-SA

4.2.1 Raw

The raw document endpoint (`https://pan-finder.panosc.ess.eu/document/raw/<doi>`) returns the raw metadata collected from the data source in json format given the doi of the document. The json document returned represent the information collected before it has been ingested and further curated by the PaN-Finder ingestor. This endpoint accept a GET request and the doi is embedded in the URL.

As an example, if we would like the data that has been collected and ingested for the doi 10.15151/ESRF-ES-52749136, the following get request should be submitted:

```
GET https://pan-finder.panosc.ess.eu/document/raw/10.15151%2FESRF-ES-52749136
```

The following will be the payload returned:


```
{
  "panosc": {
    "doi": "10.15151/ESRF-ES-52749136",
    "pid": "52749136",
    "type": "MX",
    "title": "Structural Biology at EMBL-Grenoble and IBS-CIBB",
    "endDate": "2018-07-27T08:00:00.000Z",
    "startDate": "2017-08-24T18:57:34.960Z",
    "releaseDate": "2021-07-27T08:00:00.000Z"
    ...OMITTED...
  },
  "datacite": {
    "doi": "10.15151/ESRF-ES-52749136",
    "url": "https://doi.esrf.fr/10.15151/ESRF-ES-52749136",
    "dates": [
      {
        "date": "2017-08-24",
        "dateType": "Collected"
      },
      {
        "date": "2021",
        "dateType": "Issued"
      }
    ],
    "types": {
      "resourceType": "Experiment Session",
      "resourceTypeGeneral": "Dataset"
    },
    "titles": [
      {
        "title": "Structural Biology at EMBL-Grenoble and IBS-CIBB"
      }
    ],
    ...OMITTED...
  },
  "datasets": [
    {
      "id": 112664719,
```


CC BY-SA

```
      "name": "ref-Test-Lys2_1_2359382",
      "type": "acquisition",
      "endDate": "2018-07-24T14:55:36.583+02:00",
      "sampleId": 112664718,
      "startDate": "2018-07-24T14:55:14.116+02:00",
      "parameters": [
        {
          "name": "MX_detectorDistance",
          "units": "NA",
          "value": "392.95"
        },
        {
          "name": "MX_beamSizeAtSampleY",
          "units": "NA",
          "value": "0.04"
        },
        {
          "name": "InstrumentMonochromator_wavelength",
          "units": "NA",
          "value": "0.977999953527"
        },
    ...OMITTED...
 ],
 "catalogue": {
    "id": 52749136,
    "doi": "10.15151/ESRF-ES-52749136",
    "name": "MX-1942",
    "title": "Structural Biology at EMBL-Grenoble and IBS-CIBB",
    "visitId": "id30b",
    "instrument": {
      "id": 13168154,
      "name": "ID30B",
      "instrumentScientists": []
    },
    ...OMITTED...
  }
}
```


4.2.2 Document

The document endpoint (`https://pan-finder.panosc.ess.eu/document/<doi>`) returns the relevant information of the document stored in the application database given its doi. The information returned are a subset of the information stored in the database after the raw metadata has been ingested and further curated. This endpoint accept a GET request and the doi is embedded in the URL.

Continuing with the same document used in the previous section, the request will be:

`GET https://pan-finder.panosc.ess.eu/document/10.15151%2FESRF-ES-52749136`

and the payload returned is:


```
{
```


CC BY-SA

```
 "id":9459,
 "doi":"10.15151/ESRF-ES-52749136",
 "title":"Structural Biology at EMBL-Grenoble and IBS-CIBB",
 "abstract":null,
 "facility_name":"ESRF"
}
```


The structure of the information returned might subject to changes as the project is reviewing and evaluating changes to the application frontend.

4.3 Feedback

This group allows the project to collect feedback from the user. The code is found in the `feedback.py` file in the server folder.

4.3.1 Submit

The feedback submit endpoint (`https://pan-finder.panosc.ess.eu/feedback/submit`) accept a POST request allowing the user to specify if a specific result is relevant or not within the context of a search. At moment the feedback accepted is positive or negative. It is our intention to expand th etype of feedback as the application evolves and grows.

4.4 Session

This group implements the functionalities to create and manage a session for the user. The implementation is located in the `session.py` file.

4.4.1 Create

The session create endpoint (https://pan-finder.panosc.ess.eu/session/create) accept a POST request with the cloudflare turnstyle token and returns a session id with the expiration type. This endpoint was implemented to reduce the risk of malicious usage of any endpoint. Users needs to have a valid session to be able to interact with the application.

5 Frontend (User Interface)

The PaN-Finder frontend is a javascript application based on the React framework. It was originally coded for the PaNOSC data portal and later adapted to implement the PaN-Finder application as it is currently available at the public URL https://pan-finder.panosc.ess.eu.

 CC BY-SA

In the PaN-Finder code repository, the frontend application can be found under the frontend/src folder. All the files containing the code to implement the necessary functionalities can be found in the folder frontend/src/PanFinder. All the files have been named accordingly to the component that is implemented in it.

When loaded in the browser, the frontend application check that a human is involved using a freely available cloudflare turnstile gate. We decided to include such verification to avoid abuse and reduce as much as possible automatic malicious activities.

Once the validation has been completed successfully, the user is presented with the opening page shown in figure 6.

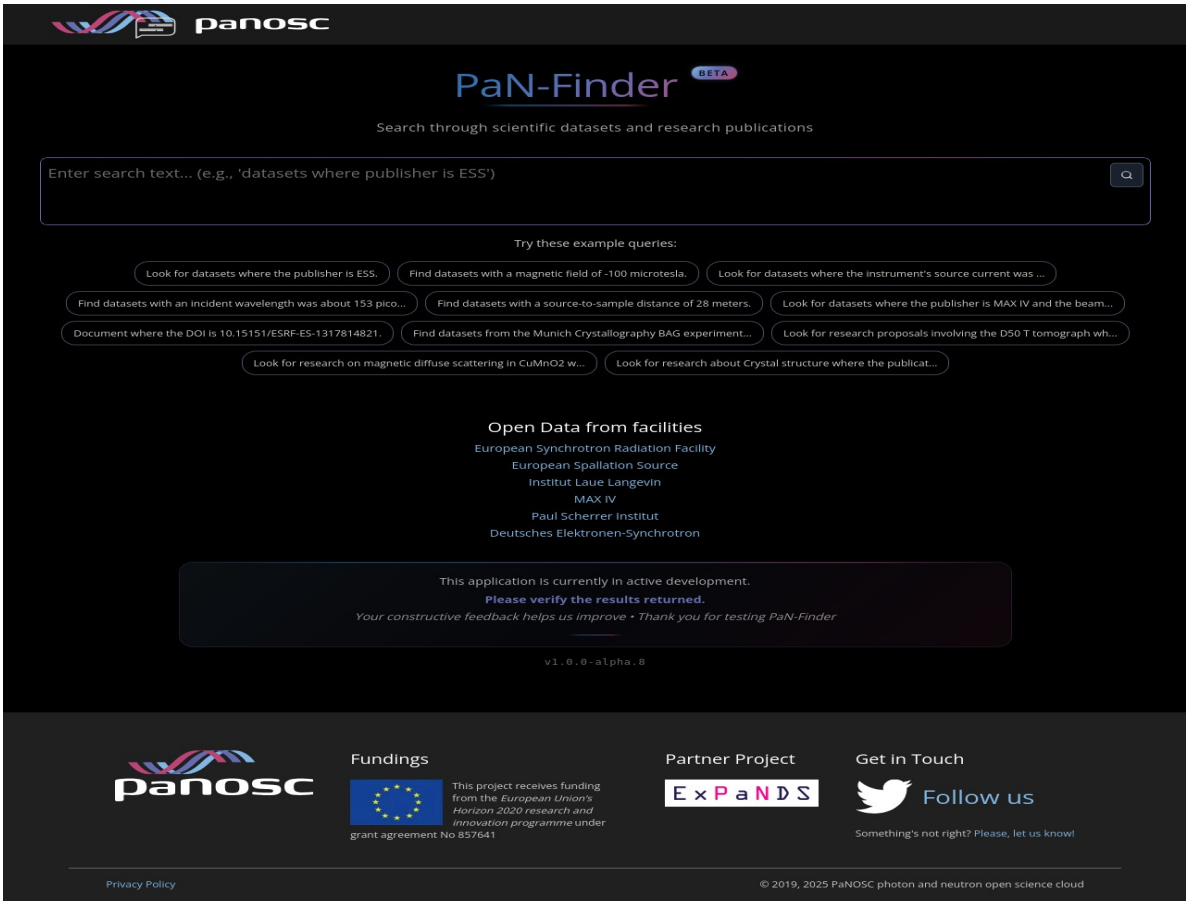

Figure 6: PaN-Finder opening page

The page present three main areas: the prompt text area, the examples list and the data sources list.

BY-SA

The user shall write their search prompt in the dedicated text area at the top part of the page. The search prompt should be a request formulated in english containing the explanation of what they are looking for and all the conditions that documents should match.
Alternatively, a novel user can select any of the listed examples placed right below of the prompt area. By clicking on one of the examples, the related search prompt is copied in the prompt area and the search is submitted to the backend.

One example listed is:

CC BY-SA

Find datasets collected with small angle scattering technique where the sample was kept at 2 kelvin

These queries examples have been selected because they provide a comprehensive set of results covering multiple use cases of relevancy, number of results present and facilities.

5.1.1 Results List

When the user submits their search, the frontend send the request to the relevant endpoint and wait for the results set. During the wait, the example list section is hidden and in its place updates on the progress are provided to the user. Once the search is completed and results have been received, they are displayed it as compact list showing minimal information for each document. At the moment of this writing, the information shown is doi, title, facility and relevance score. Only the first and most relevance results is expanded and the entry detailed view is presented with additional information, including the relevancy explanation. A screen shot of the results list presented to the user is show in figure 7.

If the user is interested in any other results, they can expand the desired document by clicking on the row. This action will expand the entry to show the entry detailed view. The relevancy explanation is generated the first time that the entry detailed view is presented.
At the time of this writing, the results list is split in two groups: the relevant results and the suggested results. This design decision is an attempt to focus the user attention to the most relevant results and comes directly from user feedback. It is still a work in progress as we are actively work on collecting user feedback and research UX best practices.

Each collapsed row offer a direct link to the facility page of the entry by embedding the source URL in the DOI presented.

Above the list of results, the frontend offer a button, labeled “Export” that allows to export the list of results to a csv file. This feature has been implemented following multiple users feedback.

CC BY-SA

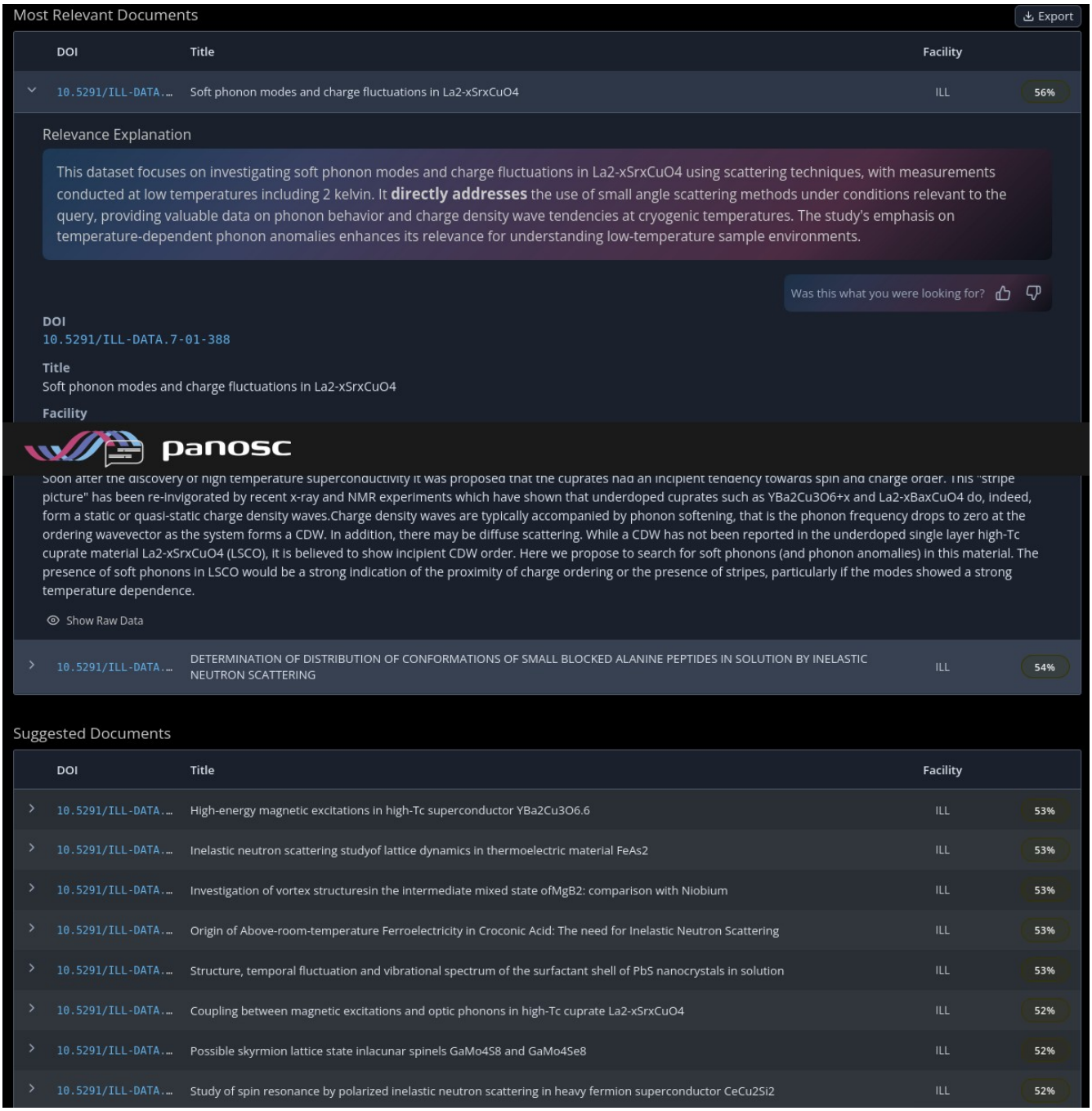

Figure 7: Search results

CC BY-SA

5.1.2 Entry detailed view

The user can decide to read more about the document before following the link to the source. They can expand the entry and view the complete “entry detailed view” which provides additional information, such as the full abstract together with an LLM generated relevancy summary explaining why the entry is relevant within the context of the user search (Figure 8).

Right under the relevancy explanation, we inserted a feedback widgets to foster results validation. Users can click thumb up or down to indicate if the specific entry is relevant to their search or not. The efficacy of this widget is still under discussion and it will receive multiple revisions before the end of the project.
The feedback collected will be used to review the accuracy of the system and included in the benchmarks.

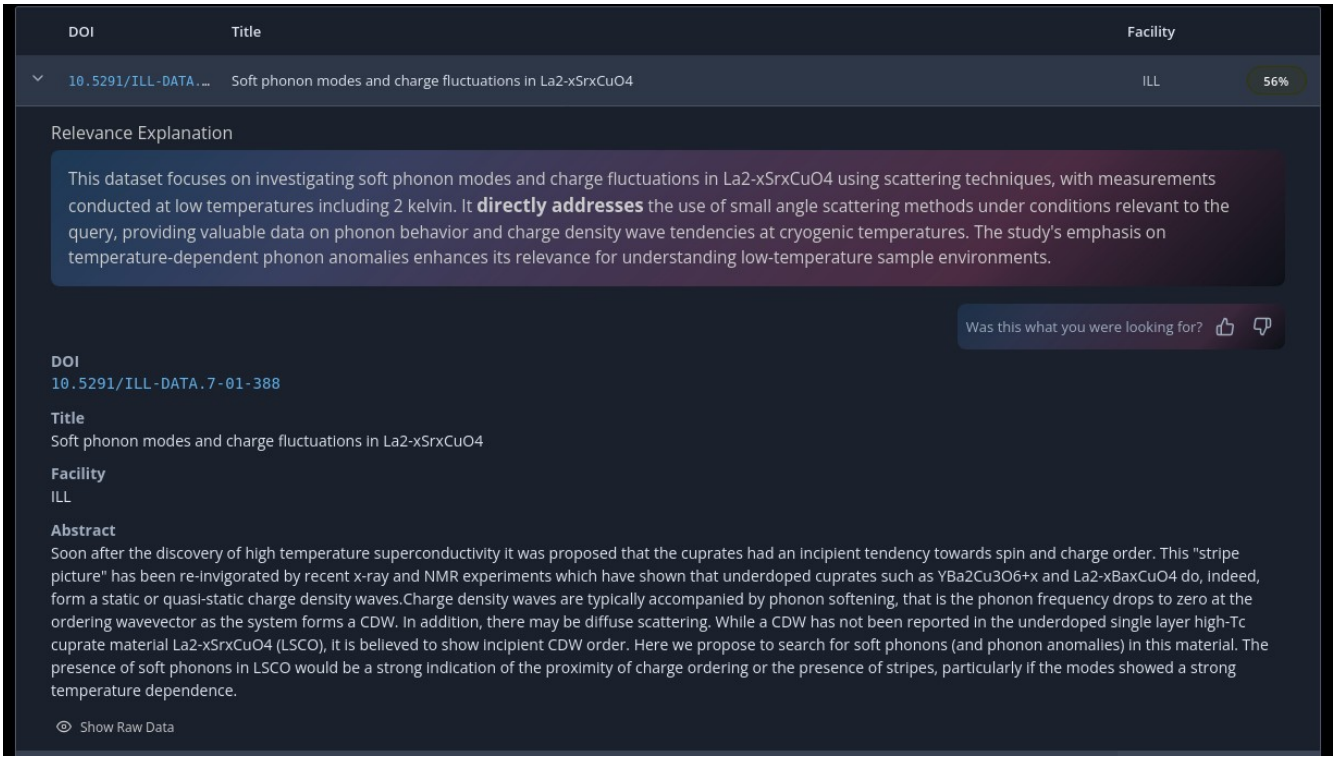

Figure 8: Entry Expanded View

The individual entry detailed view additional allows the user to view the raw metadata that have been collected from the data sources. This can be achieved by clicking on the text “`Show Raw Data`”. A click on this link will load the raw metadata from the backend and show it in json format in a dedicated

BY-SA

text area appended to the entry expanded view. The entry expanded view together with the raw metadata is visible in figure 9. Following up to users feedback "`Show Raw Data`" will be soon renamed to "`Show Raw Metadata`" to avoid confusion that a direct link to the raw scientific data is provided.

As it is visible in figure 9, it also possible to download the full entry raw metadata. This feature has been implemented to allow users to verify and validate the full entry raw metadata and also to foster reuse of this open data.

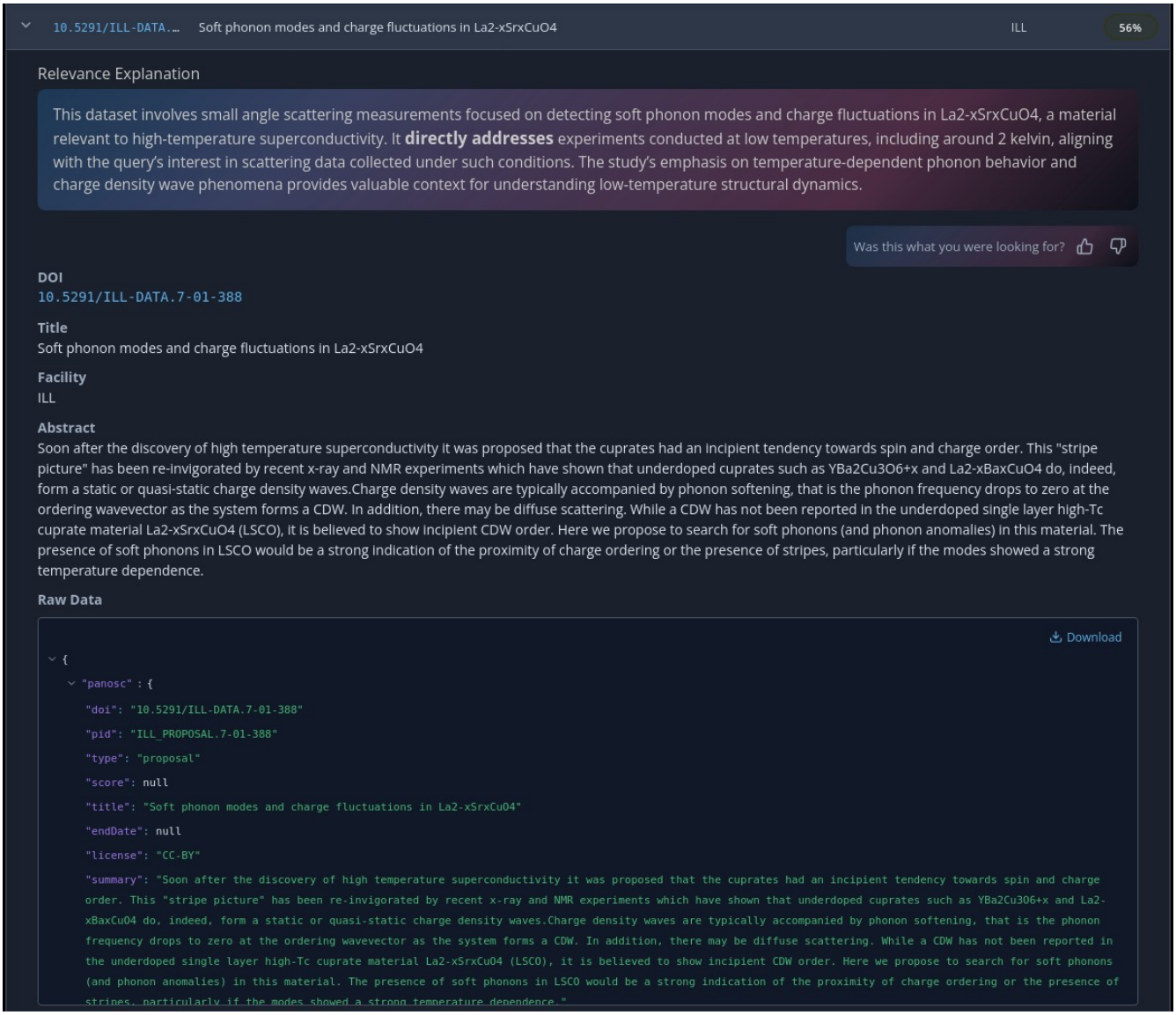

Figure 9: Entry expanded view with raw metadata

CC BY-SA

5.1.3 Query details

The user search prompt when submitted is decomposed in its components: intention, keywords and filters (See section 4.1.1 Search). We named this step the components extraction. This functionality is implemented with the aid of an LLM model which is subject to hallucination and under constant development and training. As it is not a deterministic process, we are aware that it might not capture exactly the user semantic meaning, therefore we have added the query details section in the frontend. The query details present to the user the results of the component extraction, so the user can verify the individual components (figure 10). When the extracted components are not matching exactly the user intentions, the widget offer the functionality to manually change them and submit the new query specifying directly the components and skipping the extraction step. In figure 11 the components editing is shown. Notice the search and exit button placed on the top right side of the widget. Users feedback informed us that the placement of these buttons is not optimal. We are currently working to improve UX by find a better location for them.

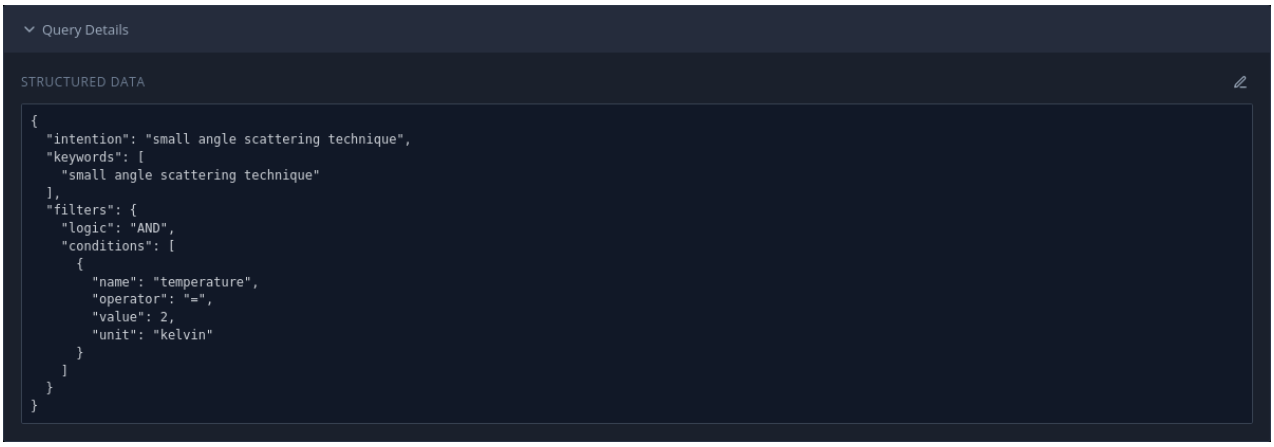

Figure 10: Query details: components

CC BY-SA

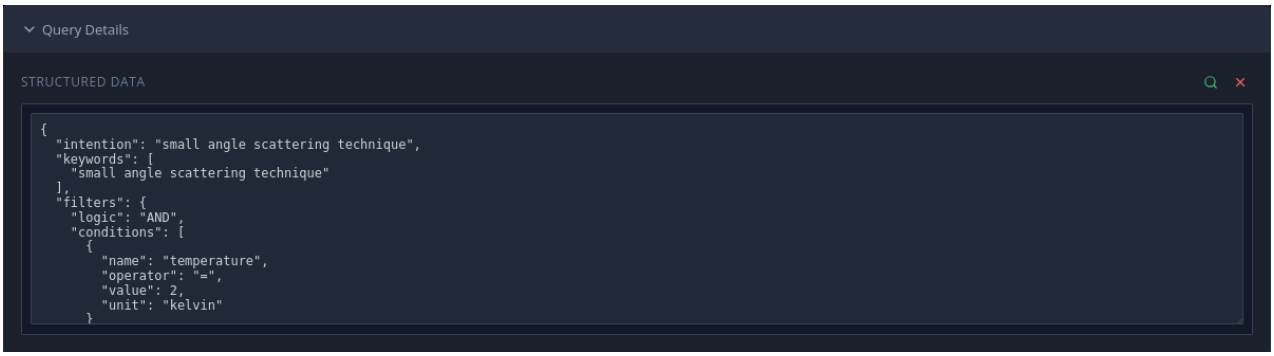

Figure 11: Query details: components editing

6 Database

In this section, we cover the database schema used in the project. The different tables will be introduce, their schema and the functionality that they have within the project. There are two different databases, the application database and the benchmark database. They will be covered separately.

6.1 Application databases

The application database is define in the file schema.sql contained in the project repository under the folder database. The sql file contains also all the functions and indexes definition, which are not covered in this deliverable. If the reader would like to find more about the full content of the schema file, we invite the user to access directly the file.

6.1.1 Document

The document table is the table where all the raw entries collected from the data sources are saved. This table also contains an AI generated summary, the embeddings vector of title and summary combined, and the postgresql text search vector of the raw input and the title and text of the entry.

An entry is a document, publication or dataset that can be retrieve by a data source which has been registered and has a unique DOI associated with it.

The schema of this table is the following:

- `id integer NOT NULL,`
  `Unique id of the entry`

CC BY-SA

- `doi text,`
  `Registered DOI of this entry`
- `title text,`
  `Title of the entry`
- `text text,`
  `Text associated with the entry. It can be the abstract or the introduction paragraph`
- `summary text,`
  `AI generated 100 words summary. This summary is generated during the ingestion process`
- `title_summary_vector public.vector(384),`
  `Embedding vector of the concatenation of the title and the summary. This information is generated during the ingestion.`
- `raw jsonb,`
  `Entry as it was collected from the the data source in json format.`
- `facility_id integer,`
  `Unique id of the facility assigned at ingestion time`
- `raw_tsvector tsvector,`
  `Text search vector of the raw entry.`
- `title_text_search_vector tsvector`
  `Text search vector of the entry title.`

6.1.2 Chunk

This table contains chunks of the text of each entry and their embeddings vector. The usable text of each entry is split in smaller portions called chunks. The chunks are created by a dedicated library at ingestion time. This table is used to perform similarity search on each entry.

The schema is:

- `id integer NOT NULL,`
  `Unique id of the chunk.`
- `document_id integer,`
  `Document id of the document this chunk has been extracted from`
- `chunk_number integer,`
  `Unique number of the chunk within the document.`
- `text text,`
  `Text of the chunk extracted from the document`

CC BY-SA

- `text_vector public.vector(384)`
  `Embeddings of the chunk used for cosine similarity`

6.1.3 Facility

This table contains the desired name of each facility.

The schema is:

- `id integer NOT NULL,`
  `Unique id for the facility`
- `name text`
  `Desired name of the facility to be used in the application`

6.1.4 Filter

This table contains all the key – value extracted from the raw entry with multiple techniques. The key value pair can be of type explicit, derived or inferred. Explicit pairs are directly contained in the raw entry. Derived pairs have been obtained by explicit ones with some additional curation and manipulation. Inferred pairs are pairs extracted from the raw entry using AI but are not explicitly declared.

The schema is:

- `id integer NOT NULL,`
  `Unique id of this key-value pair`
- `document_id integer,`
  `Document id from where this key-value pair has been extracted`
- `key text,`
  `Key of the pair`
- `value text,`
  `Original value of the pair as extracted by the raw entry.`
- `unit text,`
  `Unit of the key value pair if it is a quantity.`
- `type public.filter_type,`
  `Type of the pair. Values are: Explicit, derived and inferred.`

CC BY-SA

- `value_boolean boolean,`
  `If the original value is a boolean, the curated and correctly typed value is stored here.`
- `value_timestamp timestamp without time zone,`
  `If the original value is a timestamp, the curated and correctly typed value is stored here.`
- `value_numeric numeric,`
  `If the original value is a number, the curated and correctly typed value is stored here.`
- `value_si public.unit`
  `If the original pair is a quantity with a unit, the unit is converted to the corresponding SI unit and stored here`

6.1.5 Filter Description

This table contains the descriptions of the key for the key value pair whenever is possible. It also store the embedding value of the description.

The schema is:

- `id integer NOT NULL,`
  `Unique id for this record.`
- `filter_key_name text NOT NULL,`
  `Name of the key.`
- `description text NOT NULL,`
  `Description of the key.`
- `description_vector public.vector(384)`
  `Embedding representation of the description.`

6.1.6 Filter Key

This table contains the embedding for all the unique keys found in the key value pairs.

The schema is:

- `name text,`
  `Key name`

CC BY-SA

- `name_vector public.vector(384)`
  `Embedding vector representing the key name`

6.1.7 Statistic

This table contains the history of all the searches submitted by all the users. Each search is assigned a unique id and all the information related are saved, including user prompt, results and relevancy scores and the sql query generated. This table is useful for the application admin to study the application behavior.

The schema is:

- `id uuid DEFAULT gen_random_uuid() NOT NULL,`
  `UUID assigned by the system to each search`
- `search_query text NOT NULL,`
  `Search prompt submitted by the user`
- `structured_data jsonb NOT NULL,`
  `Results as returned by the database`
- `results jsonb NOT NULL,`
  `List of the results as shown to the user.`
- `execution_time_ms integer NOT NULL,`
  `Total execution time`
- `modified_query_id uuid,`
  `Id of the manually modified query submitted by the user`
- `created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,`
  `Creation timestamp`
- `sql_query text`
  `Full sql query created from the user prompt and run on the database to produce the results`

6.1.8 Feedback

This table contains all the user feedback collected through the application frontend after the user has submitted a search prompt.

The schema is:

- `id integer NOT NULL,`
  `Unique id of the feedback entry`

CC BY-SA

- `statistic_id uuid NOT NULL,`
  `Unique uuid of the user search. The user can submit multiple feedback on the results of each query`
- `feedback_type text NOT NULL,`
  `Feedback provided by the user. At the time of this script, the values allowed are: positive and negative.`
- `metadata jsonb,`
  `Results metadata including list of the results`
- `created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,`
  `Time when the feedback was received`

6.2 Benchmark databases

The benchmark database contains all the necessary information to run the benchmarks together with the results collected every time the benchmarks have been run.
This database is valuable for the developers and maintainers of the application to decide if a new configuration is improving accuracy or not. It also allows to have an overview of the history of the accuracy.

6.2.1 Test Pairs

This table contains all the different test pairs that are used during the benchmarks. Please see deliverable 3.b for more information on test pairs.

The schema is:

- `tpid uuid NOT NULL,`
  `Test pair unique id`
- `userprompt text,`
  `Prompt provided by the user that is submitted to the application`
- `targetdoi text,`
  `DOI of the document/entry that we are expecting to test against`
- `expectedrank integer,`
  `Expected rank of the target DOI. Not used at the moment`
- `promptid uuid,`
  `Not used`
- `expertname text,`

CC BY-SA

Name of the expert submitting the query. Used for expert test pairs, null for synthetic test pairs.

- groupid text,
  Unique id of the group the test pair has been created with.
- source public.test_pair_source,
  Source of the test pair: expert if it was created by a human domain expert, synthetic if it was created with the aid of an LLM.
- created timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
  Timestamp when this test pair has been created
- deprecated timestamp without time zone,
  Timestamp when this test pair has been deprecated and it has not been used anymore.
  All test pairs are kept for historically reasons
- type public.test_pair_type
  Type of the test pair. Originally they were only positive and negative. It has been expanded to include additional types.
- targetgroup text
  Expected group the target doi should belong to

6.2.2 Benchmarks Run

This table is used to store all the individual runs of the benchmarks with the hyperparameters used for the run.

The schema is:

- runid uuid NOT NULL,
  Unique id of the benchmark run
- startdate timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
  Timestamp when the benchmarks run has started
- enddate timestamp without time zone,
  Timestamp when the benchmarks run was completed
- hyperparameters jsonb
  Json blob with all the hyperparameters used for this run and the application configuration.

6.2.3 Benchmarks Run Test

This table contains all the information regarding an individual test within the context of a one benchmarks run.

CC BY-SA

The schema is:

- `testid uuid NOT NULL,`
  `Unique id of this test`
- `runid uuid,`
  `Unique Id of the benchmarks run this test was performed with`
- `tpid uuid,`
  `Unique id of the test pair that generated this test`
- `intention text,`
  `Prompt component intention that was extracted from the user prompt`
- `keywords text[],`
  `Prompt component keywords list that was extracted from the user prompt`
- `filters jsonb,`
  `Prompt component filters in json format that was extracted from the user prompt`
- `rank integer,`
  `Actual rank of the target document/entry`
- `runtime numeric,`
  `Total run time to perform the test expressed in seconds`
- `extractiontime numeric,`
  `Time required to perform the component extraction from the prompt expressed in seconds`
- `buildingtime numeric,`
  `Time required to build the hybrid query given the prompt components expressed in seconds`
- `querytime numeric,`
  `Time required to run the hybrid query on the database expressed in seconds`
- `sqlquery text,`
  `Full hybrid query built during the test`
- `resultsset jsonb`
  `Full result set with scores in json format`
- `ranks numeric[]`
  `In case, this is a test with multiple target doi, this is the list of the expected rank for each of target doi.`

CC BY-SA

- `actualgroups text[]`
  `In case, this is a test with multiple target doi, this list the expected group each target doi should belong to`
- `kneepointresults`
  `This field contains the results of the knee point algorithm which attempt to divide the results in relevant and suggested results`

6.2.4 Test Pair Metrics Values

This table contains the values of the metrics defined in the hyperparameters and computed for each relevant single test pair.

The schema is:

- `id uuid NOT NULL,`
  `Id for the single test pair metric value`
- `runid uuid,`
  `Unique id of the run this metric value was compute for`
- `testid uuid,`
  `Unique id of the test pair this metric value was compute for`
- `metric text,`
  `Name of the metric that was computed`
- `value numeric`
  `Value of the metric`

6.2.5 Run Metrics Values

This table contains the values of the metrics selected in the hyperparameters and computed as average of all the individual metrics across each run

The schema is:

- `id uuid NOT NULL,`
  `Id for the single run metric value`
- `runid uuid,`
  `Unique id of the run id this metric value was compute for`
- `metric text,`
  `Name of the metric that was computed`
- `value numeric`
  `Value of the metric`

CC BY-SA

7 Benchmarks

Benchmarks are covered in deliverable 3.b. Please refer to deliverable 3.a for more information

CC BY-SA

8 Funding

The PaN-Finder (Photon and Neutron federated knowledge finder) project is founded by the OSCARS project.
The authors acknowledge the OSCARS project, which has received funding from the European Commission's Horizon Europe Research and Innovation programme under grant agreement No. 101129751

9 Licensing

This document is released under ***CC BY-SA 4.0*** license.
More information about the license is available at https://creativecommons.org/licenses/by-sa/4.0.

All the software is released under **GPL-v3** license.
More information about the license is available in the LICENSE.txt file included in the repository or at the following URL https://www.gnu.org/licenses/gpl-3.0.en.html.